

Από τον κώδικα στις φόρμουλες

To `forecast.py` βήμα προς βήμα

IAM request-demand και workload forecasting

Project Optimus Version 0.1 Model walkthrough

Το έγγραφο εξηγεί τι κάνει η τρέχουσα υλοποίηση, όχι τι θα έκανε μια μελλοντική production έκδοση.

Contents

1	Τι προσπαθεί να προβλέψει ο κώδικας;	2
2	Τα τρία συστατικά του compound forecast	2
3	Πριν από το forecast.py: το workload ενός request	2
3.1	Backend tasks	3
3.2	Approval outcome και τελικό request weight	3
4	Βήμα 1: fitting του request count	4
4.1	Γιατί Negative Binomial;	4
5	Βήμα 2: από πού προκύπτει το s_{t+1};	5
5.1	To state και η σύνδεσή του με το mean	5
5.2	To score που χρησιμοποιεί η τρέχουσα υλοποίηση	5
5.3	Η ενημέρωση state	6
5.4	Αριθμητικό παράδειγμα	6
6	Βήμα 3: sampling του request count	6
7	Βήμα 4: fitting και online correction του rejection rate	7
8	Βήμα 5: workload distributions χωρίς fixed request taxonomy	8
9	Βήμα 6: η compound workload simulation	8
10	Βήμα 7: από το workload distribution στο capacity decision	9
11	Τι κάνει και τι δεν κάνει ακόμη ο τρέχων κώδικας	9
11.1	Τι κάνει	9
11.2	Τι δεν κάνει ακόμη	10
12	Συνοπτική αντιστοίχιση κώδικα–μαθηματικών	10
13	Τελική εικόνα	10

1. Τι προσπαθεί να προβλέψει ο κώδικας;

Ο κώδικας δεν προβλέπει απευθείας ένα μόνο νούμερο “συνολικού κόστους”. Κατασκευάζει πρώτα μια **κατανομή πιθανών μελλοντικών workloads**. Αυτό είναι σημαντικό: ένα capacity plan χρειάζεται όχι μόνο τη μέση ζήτηση, αλλά και μια εικόνα για την αβεβαιότητα και τις υψηλές ουρές της ζήτησης.

Για ένα μελλοντικό time bucket t (π.χ. μία ημέρα), η συνολική ζήτηση workload είναι:

$$W_t = \sum_{i=1}^{N_t} w_{i,t}$$

όπου:

- N_t είναι ο αριθμός των requests στο bucket t ,
- $w_{i,t}$ είναι το workload του request i σε capacity-unit time,
- W_t είναι το συνολικό request workload του bucket.

Το `forecast.py` παράγει πολλά πιθανά W_t μέσω simulation. Η μέση τιμή ή το P95 προκύπτουν αργότερα από αυτά τα simulated totals.

2. Τα τρία συστατικά του compound forecast

Το “compound” σημαίνει ότι το συνολικό workload αποτελείται από τυχαίο αριθμό requests, όπου κάθε request έχει τυχαίο outcome και τυχαίο βάρος. Ο κώδικας συνδυάζει τρία μοντέλα:

1. **Request-count model:** προβλέπει πόσα requests θα έρθουν, δηλαδή το N_t .
2. **Rejection-rate model:** προβλέπει ποιο ποσοστό θα απορριφθεί, δηλαδή την πιθανότητα $p_{R,t}$.
3. **Conditional workload distributions:** δίνουν το πιθανό workload ενός approved ή rejected request.

Η τελική προσομοίωση είναι επομένως:

$$N_t \longrightarrow O_{1,t}, \dots, O_{N_t,t} \longrightarrow w_{1,t}, \dots, w_{N_t,t} \longrightarrow W_t = \sum_i w_{i,t}$$

Τα βέλη σημαίνουν “κάνε ένα random draw από το προηγούμενο μοντέλο”. Δεν κάνουμε ένα αυθαίρετο άλμα από το forecast του N_t στο workload: παράγουμε ρητά τα request weights και τα αθροίζουμε.

3. Πριν από το `forecast.py`: το workload ενός request

Το `forecast.py` δεν υπολογίζει από την αρχή το κόστος κάθε ιστορικού request. Παραλαμβάνει τις κατανομές που έχουν δημιουργηθεί στο `workload.py`. Η βάση αυτών των κατανομών είναι το workload contract του `models.py`.

3.1 Backend tasks

Αν το request i δημιουργεί backend tasks $j = 1, \dots, J_i$, κάθε task έχει:

- τ_{ij} : active execution time σε δευτερόλεπτα,
- r_{ij} : capacity units που καταναλώνει όσο εκτελείται.

Το workload του task είναι το γινόμενο χρόνου επί πόρων:

$$w_{ij}^{task} = \tau_{ij} r_{ij}$$

Αν δύο tasks τρέχουν παράλληλα, τα αθροίζουμε. Ο λόγος είναι ότι μπορεί να μην αυξάνεται το wall-clock duration του request, αλλά δεσμεύονται ταυτόχρονα δύο capacity-consuming executions.

Ο κώδικας υπολογίζει επίσης προαιρετικό orchestration/workflow overhead:

$$w_i^{wf} = T_i^{wf} r_i^{wf}$$

όπου T_i^{wf} είναι ο ενεργός χρόνος του workflow engine και r_i^{wf} η ένταση κατανάλωσης capacity. Ο χρόνος αναμονής για manual approval δεν μπαίνει εδώ, εκτός αν η συγκεκριμένη αναμονή κρατά πράγματι δεσμευμένο capacity resource.

Άρα το backend μέρος είναι:

$$w_i^{backend} = \sum_{j=1}^{J_i} \tau_{ij} r_{ij} + T_i^{wf} r_i^{wf}$$

3.2 Approval outcome και τελικό request weight

Το I_i^A είναι indicator μεταβλητή: παίρνει την τιμή 1 όταν το request εγκρίνεται και 0 όταν απορρίπτεται.

$$I_i^A = \begin{cases} 1, & \text{αν το request εγκρίθηκε,} \\ 0, & \text{αν το request απορρίφθηκε.} \end{cases}$$

Αν υπάρχει κόστος μέχρι να ληφθεί η απόφαση, το ονομάζουμε w_i^{pre} . Το συνολικό observed weight είναι:

$$w_i = w_i^{pre} + I_i^A w_i^{backend}$$

Έτσι ένα rejected request διατηρεί το pre-decision κόστος, αλλά δεν παίρνει backend calculation cost. Αυτή ακριβώς η διάκριση συνδέει αργότερα το rejection forecast με το capacity forecast.

4. Βήμα 1: fitting του request count

Η κλάση `NegativeBinomialModel` ξεκινά από ιστορικούς αριθμούς requests ανά time bucket. Για παρατηρήσεις $n_1, \dots, n_T$ υπολογίζει πρώτα τον μέσο αριθμό requests:

$$\hat{\mu} = \frac{1}{T} \sum_{t=1}^T n_t$$

και τη διακύμανση με τον ίδιο population-style παρονομαστή που χρησιμοποιεί ο κώδικας:

$$\hat{v} = \frac{1}{T} \sum_{t=1}^T (n_t - \hat{\mu})^2$$

4.1 Γιατί Negative Binomial;

Για Negative Binomial με mean μ και dispersion ϕ χρησιμοποιούμε την παραμετροποίηση:

$$\text{Var}(N_t) = \mu + \phi\mu^2$$

Το πρώτο μέρος, μ , είναι η διακύμανση του Poisson. Το δεύτερο μέρος, $\phi\mu^2$, είναι επιπλέον μεταβλητότητα. Επομένως το ϕ επιτρέπει στα request counts να είναι πιο ασταθή από ένα Poisson process.

Λύνουμε την παραπάνω σχέση ως προς ϕ :

$$\begin{aligned} v &= \mu + \phi\mu^2, \\ v - \mu &= \phi\mu^2, \\ \hat{\phi} &= \frac{\hat{v} - \hat{\mu}}{\hat{\mu}^2}. \end{aligned}$$

Αυτό είναι το method-of-moments fitting που εφαρμόζει το `NegativeBinomialModel.fit`. Αν $\hat{v} \leq \hat{\mu}$, η εκτίμηση θα ήταν αρνητική, κάτι που δεν επιτρέπεται ως dispersion. Ο κώδικας την κόβει στο μηδέν:

$$\hat{\phi} = \max\left(0, \frac{\hat{v} - \hat{\mu}}{\hat{\mu}^2}\right)$$

Με $\phi = 0$ το μοντέλο γίνεται το Poisson limit. Άρα η υλοποίηση δεν επιβάλλει υπερδιασπορά όταν τα δεδομένα δεν την υποστηρίζουν.

Σημαντικό: αυτό είναι αρχικό baseline fitting, όχι πλήρης maximum likelihood optimization. Το πλεονέκτημα είναι ότι η λογική μένει ορατή και χωρίς numerical dependency. Σε επόμενο στάδιο μπορεί να αντικατασταθεί ο εκτιμητής χωρίς να αλλάξει το interface του compound forecast.

5. Βήμα 2: από πού προκύπτει το s_{t+1} ;

Αυτό είναι το σημείο που χρειάζεται τη μεγαλύτερη διευκρίνιση. Η σχέση

$$s_{t+1} = \omega + \text{persistence } s_t + \text{learning_rate score}_t$$

δεν προκύπτει αυτόματα μόνο και μόνο επειδή επιλέξαμε Negative Binomial. Είναι ένας **επιλεγμένος δυναμικός μηχανισμός online correction**, βασισμένος στη λογική των score-driven models. Ο σκοπός του είναι να ανανεώνει το forecast όταν κλείνει ένα νέο time bucket, χωρίς να ξανακάνει fit όλο το ιστορικό από την αρχή.

5.1 Το state και η σύνδεσή του με το mean

Το state s_t δεν είναι workload ούτε αριθμός requests. Είναι latent level στη λογαριθμική κλίμακα. Η σύνδεση είναι:

$$s_t = \log(\mu_t), \quad \mu_t = \exp(s_t)$$

Χρησιμοποιούμε το log διότι το μ_t πρέπει να είναι μη αρνητικό. Όποια πραγματική τιμή και αν έχει το s_t , η εκθετική μετατροπή παράγει θετικό forecast mean. Στον κώδικα το αρχικό state μετά το fit είναι:

$$s_1 = \log(\hat{\mu})$$

και το `forecast()` επιστρέφει $\exp(s_t)$.

5.2 Το score που χρησιμοποιεί η τρέχουσα υλοποίηση

Με forecast mean μ_t και observed count N_t , το raw forecast error είναι $N_t - \mu_t$. Ο κώδικας το κανονικοποιεί ως:

$$\text{score}_t = \frac{N_t - \mu_t}{1 + \phi \mu_t}$$

Η ερμηνεία είναι απλή:

- αν $N_t > \mu_t$, το score είναι θετικό και το επόμενο forecast πρέπει να κινηθεί προς τα πάνω,
- αν $N_t < \mu_t$, το score είναι αρνητικό και το επόμενο forecast πρέπει να κινηθεί προς τα κάτω,
- όσο μεγαλύτερη είναι η προβλεπόμενη dispersion, τόσο πιο ήπια είναι η διόρθωση, επειδή ένα μεγάλο error είναι λιγότερο surprising.

Το score της τρέχουσας έκδοσης είναι **normalized forecast-error score**. Δεν είναι ο ακριβής derivative της Negative Binomial log-likelihood. Αυτό πρέπει να παραμείνει ρητό: η φόρμουλα είναι πρακτική και σταθερή online διόρθωση, ενώ ένα αυστηρό GAS implementation θα χρησιμοποιούσε το scaled likelihood score της επιλεγμένης Negative Binomial parameterization.

5.3 Η ενημέρωση state

Τώρα η εξίσωση γράφεται με τα ονόματα των μεταβλητών του κώδικα:

$$s_{t+1} = \omega + \beta s_t + \alpha \text{score}_t$$

με την αντιστοίχιση:

Σύμβολο	Κώδικας	Ρόλος
ω	omega	σταθερό intercept / baseline drift
β	persistence	πόσο διατηρούμε το προηγούμενο level
α	learning_rate	πόσο αντιδρούμε στο νέο error
s_t	state	latent log-mean του τρέχοντος forecast

Η εξίσωση προκύπτει λοιπόν από τρία σχεδιαστικά desiderata:

1. Θέλουμε baseline term: ω .
2. Θέλουμε μνήμη: το παλιό level s_t να μην ξεχνιέται αμέσως, άρα βs_t .
3. Θέλουμε correction από τα νέα δεδομένα: αscore_t .

Η τρέχουσα κλάση έχει default $\beta = 0.90$, $\alpha = 0.10$ και $\omega = 0$. Αυτές οι τιμές είναι configuration defaults του prototype, όχι παράμετροι που έχουν ήδη fitted από production data. Ειδικά το $\omega = 0$ σημαίνει ότι δεν έχει προστεθεί ακόμη ξεχωριστό long-run level calibration. Για production χρήση θα πρέπει είτε να εκτιμηθεί το intercept είτε να οριστεί ρητά το μακροχρόνιο επίπεδο που θέλουμε να διατηρεί το state.

5.4 Αριθμητικό παράδειγμα

Έστω ότι σε ένα bucket:

$$\mu_t = 100, N_t = 120, \phi = 0.02, \omega = 0, \beta = 0.90, \alpha = 0.10.$$

Τότε:

$$\begin{aligned} \text{score}_t &= \frac{120 - 100}{1 + 0.02 \cdot 100} = \frac{20}{3} \approx 6.67, \\ s_{t+1} &= 0 + 0.90s_t + 0.10 \cdot 6.67 = 0.90s_t + 0.667, \\ \mu_{t+1} &= \exp(s_{t+1}). \end{aligned}$$

Το νέο bucket ήταν μεγαλύτερο από το forecast, άρα η διόρθωση αυξάνει το log-mean. Η αύξηση δεν είναι ίση με 20 requests: περνά από normalization και learning rate, ώστε ένα μόνο bucket να μην αλλάζει απότομα ολόκληρο το μοντέλο.

6. Βήμα 3: sampling του request count

Για να δημιουργήσει ένα πιθανό μελλοντικό bucket, η μέθοδος `sample_count` χρησιμοποιεί τη Gamma-Poisson mixture representation της Negative Binomial.

Για dispersion $\phi > 0$:

$$\Lambda_t \sim \text{Gamma} \left(k = \frac{1}{\phi}, \theta = \phi \mu_t \right),$$

$$N_t \mid \Lambda_t \sim \text{Poisson}(\Lambda_t).$$

Το Λ_t είναι latent Poisson rate. Η Gamma κατανομή κάνει το rate να μεταβάλλεται από simulation σε simulation. Αυτή η επιπλέον αβεβαιότητα είναι ακριβώς που δημιουργεί την υπερδιασπορά.

Πράγματι, για τη Gamma επιλογή:

$$E[\Lambda_t] = k\theta = \mu_t,$$

$$\text{Var}(\Lambda_t) = k\theta^2 = \phi \mu_t^2.$$

Με τον νόμο total variance:

$$\begin{aligned} \text{Var}(N_t) &= E[\text{Var}(N_t \mid \Lambda_t)] + \text{Var}(E[N_t \mid \Lambda_t]) \\ &= E[\Lambda_t] + \text{Var}(\Lambda_t) \\ &= \mu_t + \phi \mu_t^2. \end{aligned}$$

Όταν $\phi = 0$, δεν υπάρχει Gamma mixing layer και ο κώδικας κάνει απευθείας Poisson draw με rate μ_t . Η συνάρτηση `_sample_poisson` χρησιμοποιεί τον απλό Knuth prototype sampler για να μείνει το παράδειγμα dependency-free.

7. Βήμα 4: fitting και online correction του rejection rate

Η κλάση `RejectionRateModel` προβλέπει την πιθανότητα rejection, όχι το workload. Από D_t αποφάσεις, εκ των οποίων R_t rejected, το αρχικό estimate είναι:

$$\hat{p}_R = \frac{R_t}{D_t}$$

Η πιθανότητα αποθηκεύεται σε log-odds state q_t :

$$q_t = \log \left(\frac{p_{R,t}}{1 - p_{R,t}} \right),$$

$$p_{R,t} = \frac{1}{1 + e^{-q_t}}.$$

Το logistic transformation εγγυάται ότι το forecast probability βρίσκεται πάντα στο $(0, 1)$.

Μετά το κλείσιμο του bucket, το binomial forecast error είναι:

$$\text{score}_t^R = R_t - D_t p_{R,t}$$

και η online correction είναι:

$$q_{t+1} = \beta_R q_t + \alpha_R \text{score}_t^R$$

Αν απορρίφθηκαν περισσότερα requests από όσα αναμένονταν, το score είναι θετικό και αυξάνει το rejection probability. Στην compound simulation ένα rejected request παίρνει distribution rejected, ενώ ένα approved request παίρνει distribution approved.

8. Βήμα 5: workload distributions χωρίς fixed request taxonomy

Οι παρατηρήσεις του `workload.py` χωρίζονται μόνο με βάση το outcome:

$F_A(w)$: empirical distribution workload για approved requests,
 $F_R(w)$: empirical distribution workload για rejected requests.

Δεν απαιτείται προκαθορισμένη κατηγορία τύπου request. Τα διαφορετικά backend tasks, durations και capacity multipliers αποτυπώνονται απευθείας στα observed weights w_i .

Για να έχει πρόσφατο ιστορικό μεγαλύτερη επιρροή, η παρατήρηση ηλικίας a παίρνει weight:

$$\rho_a = \lambda^a, \quad 0 < \lambda \leq 1$$

Το ρ_a είναι **στατιστικό recency weight**, όχι capacity unit. Στο code το νεότερο observation έχει $a = 0$ και weight 1, ενώ παλαιότερα observations έχουν weight λ^a . Η weighted empirical distribution δεν υποθέτει Normal κατανομή και κρατά την πραγματική ασυμμετρία και τα ακριβά tails των request weights.

Ο weighted μέσος είναι:

$$\bar{w}_A = \frac{\sum_{i \in A} \rho_i w_i}{\sum_{i \in A} \rho_i}$$

και αντίστοιχα για rejected requests. Για simulation, το `WeightedEmpiricalDistribution.sample` επιλέγει ένα ιστορικό observed weight με πιθανότητες ανάλογες των ρ_i .

9. Βήμα 6: η compound workload simulation

Η `compound_workload_forecast` επαναλαμβάνει τα παρακάτω βήματα για M simulations:

1. Κάνει draw request count: $N^{(m)} \sim \text{NB}(\mu_t, \phi)$.
2. Για κάθε request $k = 1, \dots, N^{(m)}$, κάνει outcome draw: $O_k^{(m)} \sim \text{Bernoulli}(1 - p_{R,t})$ για approval.
3. Επιλέγει workload weight από F_A αν το outcome είναι approved και από F_R αν είναι rejected.
4. Αθροίζει όλα τα weights: $W^{(m)} = \sum_{k=1}^{N^{(m)}} w_k^{(m)}$.

Το αποτέλεσμα δεν είναι ένας αριθμός αλλά ordered sample:

$$W^{(1)}, W^{(2)}, \dots, W^{(M)}$$

Από αυτό μπορούμε να υπολογίσουμε:

- expected workload: περίπου $\frac{1}{M} \sum_m W^{(m)}$,
- P50: τυπικό/median workload,
- P95: workload που καλύπτει περίπου το 95% των simulated scenarios.

Το P95 είναι αυτό που χρησιμοποιεί στη συνέχεια το `capacity.py` ως protected request capacity. Άρα η διαδρομή είναι:

count forecast + rejection forecast + weight distributions $\implies$ distribution of total workload

10. Βήμα 7: από το workload distribution στο capacity decision

Αν C_t^{total} είναι το capacity που υπάρχει στο bucket, και $Q_{0.95}(W_t)$ το empirical 95th percentile των simulated workloads, τότε:

$$\begin{aligned} C_t^{requests} &= Q_{0.95}(W_t), \\ C_t^{spare} &= \max(0, C_t^{total} - C_t^{requests}), \\ C_t^{deficit} &= \max(0, C_t^{requests} - C_t^{total}). \end{aligned}$$

Το request capacity έχει προτεραιότητα επειδή δεσμεύεται πρώτα το $C_t^{requests}$. Άλλα objects μπορούν να χρησιμοποιήσουν μόνο το C_t^{spare} .

11. Τι κάνει και τι δεν κάνει ακόμη ο τρέχων κώδικας

11.1 Τι κάνει

- Κάνει method-of-moments baseline fit για το request count.
- Επιτρέπει overdispersion μέσω Negative Binomial και Poisson limit.
- Κάνει online state correction μετά από νέο observed bucket.
- Μοντελοποιεί rejection ως ξεχωριστή probabilistic διαδικασία.
- Διατηρεί empirical request-weight distributions αντί να επιβάλλει Normal υπόθεση.
- Παράγει predictive distribution του συνολικού workload μέσω Monte Carlo compound simulation.

11.2 Τι δεν κάνει ακόμη

- Η παράμετρος `horizon` υπάρχει στο interface, αλλά η τρέχουσα υλοποίηση επιστρέφει το ίδιο state για κάθε horizon. Δεν υπάρχει ακόμη multi-step state propagation.
- Το `score` είναι normalized error και όχι exact Negative Binomial likelihood score.
- Τα `persistence`, `learning_rate` και `omega` δεν έχουν fitted από production history.
- Ο sampler Poisson είναι αναγνώσιμος prototype και όχι ο αποδοτικότερος sampler για πολύ μεγάλα rates.
- Δεν υπάρχει ακόμη queue/backlog state ή service-level model. Το παρόν βήμα προβλέπει demand και protected capacity.

12. Συνοπτική αντιστοίχιση κώδικα–μαθηματικών

Στοιχείο κώδικα	Μαθηματικός ρόλος
NegativeBinomialModel.fit	$\hat{\mu}$ και $\hat{\phi}$ από moments
forecast	$\mu_t = \exp(s_t)$
update	$s_{t+1} = \omega + \beta s_t + \alpha \text{score}_t$
sample_count	Gamma–Poisson mixture
RejectionRateModel.fit	$\hat{p}_R = R/D$ και log-odds state
RejectionRateModel.update	binomial score correction
fit_workload_distributions	recency-weighted F_A, F_R
compound_workload_forecast	$W^{(m)} = \sum_k w_k^{(m)}$
empirical_quantile	$Q_p(W)$ από simulated totals

13. Τελική εικόνα

Ο κώδικας ακολουθεί την εξής αλυσίδα:

1. Τα IAM events μετατρέπονται σε request observations.
2. Κάθε observation μετατρέπεται σε workload weight με βάση τον ενεργό χρόνο, τα capacity units, τα backend tasks και το approval outcome.
3. Τα ιστορικά counts χρησιμοποιούνται για να εκτιμηθούν το mean και η dispersion της ζήτησης.
4. Το state s_t μεταφέρει το forecast από bucket σε bucket και το score διορθώνει το state όταν έρχεται νέα πραγματικότητα.
5. Η προσομοίωση συνδυάζει τον αριθμό requests, το rejection rate και τα empirical weights.
6. Το simulated total-workload distribution μετατρέπεται σε protected request capacity, spare capacity και deficit.

Η εξίσωση του s_{t+1} είναι επομένως ο μηχανισμός που επιτρέπει στο count forecast να μην είναι στατικό: διατηρεί μνήμη, βλέπει το νέο error και κάνει ελεγχόμενη διόρθωση. Δεν υπολογίζει από μόνη της το workload. Το workload προκύπτει μόνο στο επόμενο στάδιο, όταν το διορθωμένο count forecast συνδυάζεται με outcomes και per-request weight distributions.